SkillTrend AI

What should I learn next — and what am I wasting time on?

> Most "trending skills" lists tell you what is popular. They do not tell you what to do about it. **SkillTrend AI analyses 58,276 real job postings and answers three decisions instead:** which tools in your stack are redundant, which in-demand skills you are missing, and where each skill's demand is heading.
>
> Every answer shows the numbers behind it. Upload your own postings and the forecast re-fits on your data.

What it actually outputs

Give it the stack you already know:

```
Tableau · Power BI · Python · SQL

REDUNDANT  2 tools in your stack do the same job: Tableau, Power BI
           Keep Power BI — 27.5% of postings, rising (+2.02p).
           Dropping Tableau costs you no coverage.

MISSING    5 skills commonly paired with your stack
           Statistical Analysis — in 52% of postings that want Python
           R — in 51%   ·   Excel — in 46% of postings that want Power BI

DECLINING  nothing in your stack is on the way out
```

Features

Decide	Enter your stack → redundancy verdicts, missing skills, and declining warnings, each with its supporting percentage. Includes browsable **substitute families** (e.g. Tableau / Power BI / Looker) with a recommended member.
Forecast	6-month projection per skill with an 80% uncertainty band — and its own **backtested accuracy** displayed alongside, including when the model loses to a naive baseline.
Upload	Drop in a CSV of job postings. Columns are auto-detected. **Append** splices onto the historical corpus so the forecast origin moves to your latest month; **standalone** analyses your file alone.
Roles	What 9 role families actually ask for, built from a live monthly feed of current postings — a separate, present-day data source.
Method	How every number was produced, and where it fails. Limitations are a page, not a footnote.

Design rule: the app will not show a prediction without the evidence for trusting it. Upload fewer than 12 months of history and it *refuses* to forecast, with the reason, rather than drawing a confident line through noise.

How it works

The heavy work runs **offline** and writes Parquet files. The API only reads them. Fitting 88 forecasting models cannot happen inside an HTTP request — so training and serving are separated, the same split real ML products use.

Offline pipeline

1	**Load** — inspect schema, date coverage, nulls before trusting anything	`61,953 raw`
2	**Clean** — parse dates, drop duplicates and stub descriptions	`58,276 usable`
3	**Extract** — spaCy PhraseMatcher over a 120-skill dictionary	`~20 sec`
4	**Aggregate** — skill × month as a *share* of postings + month-quality gate	`skill_monthly`
5	**Trends** — rolling averages, rising / stable / declining	`trends`
6	**Forecast** — Prophet, 6 months, backtested vs a naive baseline	`88 models`
7	**Co-occurrence** — PPMI skill graph + its validation suite	`graph`
8	**Relations** — curated families ranked by live + historical demand	`families`

Live track — scheduled monthly, no key required

9	**Fetch** — Arbeitnow · Jobicy · Remotive job-board APIs	`live_postings`
10	**Roles** — classify titles into role families, extract skills per role	`role profiles`

Serving

`FastAPI` loads the Parquet artifacts once at boot and serves 13 endpoints; the React app calls them. Only the upload path computes anything on demand — it cleans, extracts and fits a fresh forecast against your file.

Two decisions worth explaining

Percentages, not counts

Monthly posting volume swings from 3,646 down to 272. Raw counts would show every skill collapsing in late 2024 — when only the data collection changed. Measuring each skill as a share of that month's postings makes months comparable.

A dictionary, not an LLM

Deterministic, auditable, free, and 20 seconds over 58,000 documents. Regex was rejected because `\b` breaks on `C++`, `C#` and `CI/CD`; PhraseMatcher tokenises patterns and text identically. Three rules keep it precise: longest match wins, ambiguous names never match alone (`Go` is also a verb), and context blocklists reject "excel *at* communication".

Result: 82.2% of postings yielded a skill, against 78.6% for the labels shipped with the dataset — using a *smaller* vocabulary (120 terms vs 134).

Tech stack

LAYER	USED	WHY
Data & NLP	Python · pandas · **spaCy** (PhraseMatcher) · PyArrow / Parquet	Tokeniser-only spaCy — no model download. Parquet keeps artifacts small enough to commit.
Modelling	**Prophet** + a naive carry-forward baseline · PPMI co-occurrence	Every forecast is scored against the baseline, so a weak model cannot hide.
Backend	**FastAPI** · Pydantic · Uvicorn	Typed responses, auto-generated API docs, artifacts loaded once at startup.
Frontend	**React** · Vite · Tailwind · Recharts · React Router	Fast build, charts that render identically in a screenshot and in the browser.
Automation	GitHub Actions (monthly cron)	The live feed accumulates history on its own, free.
Deployment	Render (API) · Vercel (frontend) — both free tier	Serving artifacts are committed, so a deploy clones and starts. No pipeline run, no database.

Data sources — three, never mixed

	HISTORICAL	LIVE	YOUR UPLOAD
Source	Kaggle — Google job search scrape	3 job-board APIs	your CSV
Size	58,276 postings	~4,500, growing monthly	up to 200,000 rows
Span	Nov 2022 – Jul 2024	Nov 2026 onward	yours
Roles	Data analyst only	9 role families	yours
Powers	trends, forecasts, co-occurrence	role comparison, current demand	your own forecast

Averaging a 2022–24 US analyst corpus with a 2026 remote-European feed would produce a number describing nobody — so every response states which source it came from.

What this project is honest about

- **A hypothesis failed, and the failure stayed in the code.** I expected substitutes to *repel* statistically. Job ads write "Tableau, Power BI *or* Looker", so alternatives co-occur constantly — substitutes and complements measured *identically* on every metric tried. Substitution is therefore curated by hand; ranking and gap-finding stay fully data-driven.
- **Forecasts barely beat a naive baseline** — Prophet wins on 53% of skills. Each forecast reports its own result, including its losses.
- **One role, one source** in the historical data, and **a dictionary cannot find what it does not list** — a skill outside the 120 reads as 0%.

The catch that mattered most. An early trend report showed 29 skills declining and only 2 rising — a dramatic-looking finding. It was the scraper truncating description text, not the market moving. A month-quality gate now excludes affected months from every conclusion while still drawing them, shaded, on the charts.